WEEK-8

🖥️ Automation of System Tasks with Shell Scripts

1. Why Automate?

- Saves time and reduces repetitive manual work.
- Ensures consistency and accuracy.
- Useful for system administration, backups, monitoring, and routine tasks.

2. Common Tasks to Automate

- **File management**: create, copy, move, delete files.
- **System monitoring**: check disk usage, memory, CPU load.
- **Backups**: compress and archive files regularly.
- **User management**: add/remove users, set permissions.
- **Scheduling tasks**: run scripts automatically using `cron`.

3. Basic Automation Script Examples

a) Disk Usage Monitoring

Bash Copy

```bash
#!/bin/bash
# Check disk usage and alert if above threshold

threshold=80
usage=$(df -h / | grep '/' | awk '{print $5}' | sed 's/%//')

if [ $usage -gt $threshold ]
then
    echo "Warning: Disk usage is above $threshold% (Current: $usage%)"
else
    echo "Disk usage is normal ($usage%)"
fi
```

📃 Detailed Explanation

1. `#!/bin/bash`

- This is the **shebang (hashbang)** line.
- It tells the system to use the **Bash shell** to interpret the script.

2. `threshold=80`

- Defines a variable called `threshold` with value `80`.
- This means if disk usage goes above **80%**, the script will issue a warning.

3. `usage=$(df -h / | grep '/' | awk '{print $5}' | sed 's/%//')`

This is the most complex line — let's break it down step by step:

- `df -h /`
  - `df` = disk free command (shows disk usage).
  - `-h` = human-readable format (e.g., GB, MB).
  - `/` = check the root filesystem.
- `grep '/'`
  - Filters the output to only show the line containing `/` (the root partition).
- `awk '{print $5}'`
  - `awk` is a text processing tool.
  - `{print $5}` means: print the **5th column** of the line.
  - In `df` output, the 5th column is the **percentage of disk used** (e.g., `85%`).
- `sed 's/%//'`
  - `sed` is a stream editor.
  - `s/%//` removes the `%` symbol from the usage value.
  - So `85%` becomes `85`.
- `usage=$(...)`
  - Stores the final numeric value (e.g., `85`) into the variable `usage`.

4. `if [ $usage -gt $threshold ]`

- Checks if the current disk usage (`usage`) is **greater than** (`-gt`) the threshold (80).
- If true → run the commands inside the `then` block.
- If false → run the commands inside the `else` block.

5. `echo "Warning: Disk usage is above $threshold% (Current: $usage%)"`

- Prints a warning message if usage exceeds threshold.
- `$threshold` and `$usage` are replaced with their values.

6. `echo "Disk usage is normal ($usage%)"`

- Prints a normal status message if usage is below threshold.

b) Automatic Backup

Bash Copy

```bash
#!/bin/bash
# Backup a directory into a compressed file

src="/home/user/documents"
dest="/home/user/backup"
filename="backup_$(date +%Y%m%d).tar.gz"

tar -czf $dest/$filename $src
echo "Backup created at $dest/$filename"
```

🧾 Detailed Explanation

1. `#!/bin/bash`

- The **shebang (hashbang)** line.
- Tells the system to run this script using the **Bash shell**.

2. `src="/home/user/documents"`

- Defines the **source directory** you want to back up.
- Here, it's `/home/user/documents`.

3. `dest="/home/user/backup"`

- Defines the **destination directory** where the backup file will be stored.
- Here, it's `/home/user/backup`.

4. `filename="backup_$(date +%Y%m%d).tar.gz"`

- Creates a **dynamic filename** for the backup.
- `$(date +%Y%m%d)` → inserts today's date in `YYYYMMDD` format (e.g., `20260304` for March 4, 2026).
- So the file will look like: `backup_20260304.tar.gz`.

5. `tar -czf $dest/$filename $src`

- This is the **backup command**:
  - `tar` → archive utility.
  - `-c` → create a new archive.
  - `-z` → compress using gzip.
  - `-f` → specify filename.
- `$dest/$filename` → full path of the backup file.

- `$src` → the directory being backed up.
- Example: It creates `/home/user/backup/backup_20260304.tar.gz` containing all files from `/home/user/documents`.

6. `echo "Backup created at $dest/$filename"`

- Prints a confirmation message showing where the backup file was saved.

c) System Log Cleaner

Bash Copy

```bash
#!/bin/bash
# Clean old log files

logdir="/var/log"
find $logdir -type f -name "*.log" -mtime +7 -exec rm {} \;
echo "Old log files cleaned."
```

🧾 Detailed Explanation

1. `#!/bin/bash`

- The **shebang line** tells the system to run this script using the **Bash shell**.

2. `logdir="/var/log"`

- Defines a variable `logdir` that points to the directory where log files are stored.
- `/var/log` is the standard location for system log files in Linux.

3. `find $logdir -type f -name "*.log" -mtime +7 -exec rm {} \;`

This is the core command. Let's break it down:

- `find $logdir` → search inside the `/var/log` directory.
- `-type f` → look for **files** (not directories).
- `-name "*.log"` → only match files ending with `.log`.
- `-mtime +7` → select files **modified more than 7 days ago**.
- `-exec rm {} \;` → for each file found, execute the `rm` (remove) command.
  - `{}` → placeholder for the current file found.
  - `\;` → ends the `-exec` command.

👉 In short: this command deletes all `.log` files older than 7 days in `/var/log`.

4. `echo "Old log files cleaned."`

- Prints a confirmation message after the cleanup is done.

4. Scheduling Automation

- Use **cron jobs** to run scripts automatically.
- **Syntax:**

Bash Copy

```bash
crontab -e
```

Add entry:

Code Copy

```
0 2 * * * /home/user/backup.sh
```

→ Runs backup script every day at 2 AM.

1. Monitoring Memory Consumption

- **Concept:** Use `free` or `vmstat` to check memory usage.
- **Script Example:**

Bash Copy

```bash
#!/bin/bash
# Monitor memory consumption

threshold=70
usage=$(free | grep Mem | awk '{print $3/$2 * 100.0}')

echo "Current memory usage: $usage%"

if (( ${usage%.*} > threshold ))
then
    echo "Warning: Memory usage above $threshold%"
else
    echo "Memory usage is normal."
fi
```

🧾 **Detailed Explanation**

1. `#!/bin/bash`

- Shebang line → tells the system to run this script using the **Bash shell**.

2. `threshold=70`

- Sets a **limit** (70%) for memory usage.
- If usage goes above this, the script will issue a warning.

3. `usage=$(free | grep Mem | awk '{print $3/$2 * 100.0}')`

This line calculates memory usage percentage. Let's break it down:

- `free` → shows memory usage statistics.
- `grep Mem` → filters the line containing memory info.
- `awk '{print $3/$2 * 100.0}'` → processes the line:
  - `$2` = total memory.
  - `$3` = used memory.
  - `$3/$2 * 100.0` → calculates percentage of memory used.
- `usage=$(...)` → stores the result in the variable `usage`.

👉 Example: If total memory = 8000 MB and used = 4000 MB → usage = 50%.

4. `echo "Current memory usage: $usage%"`

- Prints the calculated memory usage percentage.

5. `if (( ${usage%.*} > threshold ))`

- `${usage%.*}` → strips off the decimal part (e.g., `72.5` → `72`).
- `(( ... ))` → arithmetic comparison in Bash.
- Checks if usage > threshold (70).

6. `then ... else ... fi`

- If usage is greater than threshold → print warning.
- Otherwise → print normal status.
- `fi` → ends the `if` block.

2. Checking Remote Server Connectivity

- **Concept:** Use `ping` or `nc` (netcat) to test connectivity.
- **Script Example:**

Bash Copy

```bash
#!/bin/bash
# Check connectivity to a remote server

server="google.com"

ping -c 2 $server > /dev/null

if [ $? -eq 0 ]
then
    echo "Server $server is reachable."
else
    echo "Server $server is not reachable."
fi
```

🧾 Detailed Explanation

1. `#!/bin/bash`

- Shebang line → tells the system to run this script using the **Bash shell**.

2. `server="google.com"`

- Defines a variable called `server` with the value `google.com`.
- This is the remote server you want to check connectivity to.

3. `ping -c 2 $server > /dev/null`

- `ping` → sends ICMP packets to test if a server is reachable.
- `-c 2` → send **2 packets only** (instead of running indefinitely).
- `$server` → expands to `google.com`.
- `> /dev/null` → discards the output (so you don't see ping details, only the result matters).

👉 This command tries to ping `google.com` twice silently.

4. `if [ $? -eq 0 ]`

- `$?` → special variable that stores the **exit status** of the last command.
  - `0` means success (command executed correctly).

    - Non-zero means failure (error occurred).
- So here, if `ping` succeeds (server reachable), `$?` will be `0`.

5. `then ... else ... fi`

- If ping succeeded → print `"Server google.com is reachable."`
- If ping failed → print `"Server google.com is not reachable."`
- `fi` → ends the `if` block.

3. Running at Different Frequencies

- **Concept:** Use **cron jobs** to schedule scripts at desired intervals.
- **Cron Syntax:**

Code Copy

```
* * * * * command   # minute, hour, day, month, weekday
```

- **Examples:**
    - Run memory monitoring every 5 minutes:

Code Copy

```
*/5 * * * * /home/user/memory_monitor.sh
```

    - Check server connectivity every hour:

Code Copy

```
0 * * * * /home/user/server_check.sh
```

    - Run backup every day at midnight:

Code Copy

```
0 0 * * * /home/user/backup.sh
```

4. Scheduling Automation

- Use **cron jobs** to run scripts automatically.
- **Syntax:**

Bash Copy

```bash
crontab -e
```

Add entry:

Code Copy

```
0 2 * * * /home/user/backup.sh
```

→ Runs backup script every day at 2 AM.

Cron Command to automate System Tasks

- cron processes will run in background, so to check whether scripts are executed or not, one way to check the log file i.e. /var/log/syslog
- Search for cron in syslog using command: cat /var/log/syslog/ | grep cron
- Users can see all the entries, which shows the script executed at a scheduled time mentioned in the crontab file.

=== * ===

Simple Example:
File Name: task.sh path: /home/am2en/ (say for example)

Contents:
echo Welcome to Task Scheduler Demo
echo Try date Command
date
echo List all file/folders in a home directory
ls
echo End of Task Scheduler Demo

=== * ===

• Change the execution permission to task.sh

chmod 764 task.sh

• Add the task to /etc/crontab (Task will be executed at 4.13PM, User can Change timings according to the requirements)

13 16 * * * root sh /home/am2en/task.sh > /home/am2en/output.txt

Or

13 16 * * * .task.sh >> output.txt

• Save and Exit. And Wait till 4.13pm

• After that Check the output.txt by the command

cat /home/am2en/output.txt

• Output of the task.sh is present in output.txt.

• Alternatively, /var/log/syslog can be verified about the execution of the Task.

• Similarly you can create any shell script based on your requirement and automate it using cron.